DBMS II Project

GreenAV:

A smart farming database management system

APP URL: https://greenav.vercel.app

GitHub Repo Link: https://github.com/traceback-afk/greenav

Group members:

Seyedehsan Damak – 22311781: DDL and DML

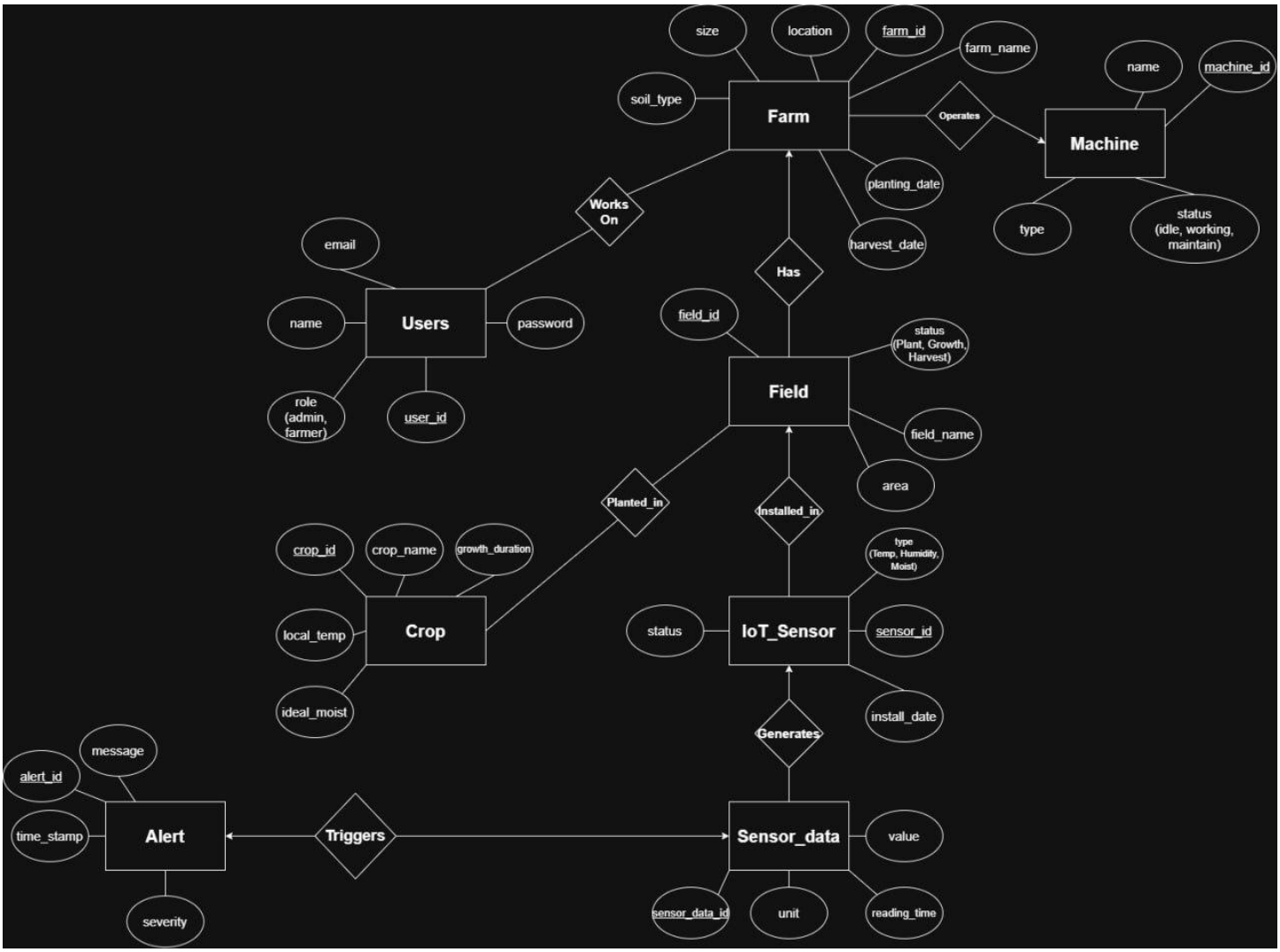
Armin Safizadeh - 22312406: ERD

Melikasadat Anvar - 22218189: Functions, Procedures and Triggers

Khashayar Khosrosourmi - 22302442: Web Application and DB connections

Introduction

GreenAV is a smart farming database management system that uses PostgreSQL (Supabase) and NextJS. We defined the following main entities: User, Farm, Field, IOT_Sensor, Crop, Sensor_Data, Alert and Machine. Users can be either Admin or Farmer, and the dashboard and permissions are different based on their role. Admins have full permission to view, insert, update and delete everything, whereas farmers can only view the farm they are assigned to, its corresponding fields, alerts and machines. The permissions are implemented inside the web application's API, where the user's role is checked. In the following ERD, you can see the main entity tables as well as junction tables:



DDL:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    name TEXT NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('admin', 'farmer'))
);

-- Farm Table (owned by admin, worked on by farmers)
CREATE TABLE farm (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    size NUMERIC NOT NULL,
    location TEXT NOT NULL,
    soil_type TEXT,
```

```
    planting_date DATE,
    harvest_date DATE
);

-- Farmer WorksOn Farm (relationship) - farmers work on farms
CREATE TABLE works_on_farm (
    farmer_id INT NOT NULL,
    farm_id INT NOT NULL,
    PRIMARY KEY (farmer_id, farm_id),
    FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (farm_id) REFERENCES farm(id) ON DELETE CASCADE
);

-- Field Table
CREATE TABLE field (
    id SERIAL PRIMARY KEY,
    farm_id INT NOT NULL,
    name TEXT NOT NULL,
    area NUMERIC NOT NULL,
    status TEXT CHECK (status IN ('Plant', 'Growth', 'Harvest')),
    FOREIGN KEY (farm_id) REFERENCES farm(id) ON DELETE CASCADE
);

-- Crop Table
CREATE TABLE crop (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    growth_duration INT NOT NULL,
    local_temp NUMERIC,
    ideal_moisture NUMERIC
);

-- Field Contains Crop (relationship)
CREATE TABLE contains (
    field_id INT NOT NULL,
    crop_id INT NOT NULL,
    PRIMARY KEY (field_id, crop_id),
    FOREIGN KEY (field_id) REFERENCES field(id) ON DELETE CASCADE,
    FOREIGN KEY (crop_id) REFERENCES crop(id) ON DELETE CASCADE
);

-- Crop Planted_in Field (relationship)
```

```sql
CREATE TABLE planted_in (
    crop_id INT NOT NULL,
    field_id INT NOT NULL,
    PRIMARY KEY (crop_id, field_id),
    FOREIGN KEY (crop_id) REFERENCES crop(id) ON DELETE CASCADE,
    FOREIGN KEY (field_id) REFERENCES field(id) ON DELETE CASCADE
);

-- Machine Table
CREATE TABLE machine (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('idle', 'working', 'maintain'))
);

-- Machine WorksOn Farm (relationship)
CREATE TABLE works_on_farm_machine (
    farm_id INT NOT NULL,
    machine_id INT NOT NULL,
    PRIMARY KEY (farm_id, machine_id),
    FOREIGN KEY (farm_id) REFERENCES farm(id) ON DELETE CASCADE,
    FOREIGN KEY (machine_id) REFERENCES machine(id) ON DELETE CASCADE
);

-- Machine WorksOn Field (relationship)
CREATE TABLE works_on_field (
    field_id INT NOT NULL,
    machine_id INT NOT NULL,
    PRIMARY KEY (field_id, machine_id),
    FOREIGN KEY (field_id) REFERENCES field(id) ON DELETE CASCADE,
    FOREIGN KEY (machine_id) REFERENCES machine(id) ON DELETE CASCADE
);

-- IoT_Sensor Table
CREATE TABLE iot_sensor (
    id SERIAL PRIMARY KEY,
    type TEXT NOT NULL CHECK (type IN ('Temp', 'Humidity', 'Moist')),
    status TEXT NOT NULL,
    install_date DATE NOT NULL,
    sensor_id TEXT UNIQUE
);
```

```sql
-- Field Installed_in IoT_Sensor (relationship)
CREATE TABLE installed_in (
    field_id INT NOT NULL,
    sensor_id INT NOT NULL,
    PRIMARY KEY (field_id, sensor_id),
    FOREIGN KEY (field_id) REFERENCES field(id) ON DELETE CASCADE,
    FOREIGN KEY (sensor_id) REFERENCES iot_sensor(id) ON DELETE CASCADE
);

-- Sensor_data Table
CREATE TABLE sensor_data (
    id SERIAL PRIMARY KEY,
    value NUMERIC NOT NULL,
    unit TEXT NOT NULL,
    reading_time TIMESTAMP NOT NULL
);

-- IoT_Sensor Generates Sensor_data (relationship)
CREATE TABLE generates (
    sensor_id INT NOT NULL,
    sensor_data_id INT NOT NULL,
    PRIMARY KEY (sensor_id, sensor_data_id),
    FOREIGN KEY (sensor_id) REFERENCES iot_sensor(id) ON DELETE CASCADE,
    FOREIGN KEY (sensor_data_id) REFERENCES sensor_data(id) ON DELETE CASCADE
);

-- Alert Table
CREATE TABLE alert (
    id SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    timestamp TIMESTAMP NOT NULL,
    severity TEXT NOT NULL CHECK (severity IN ('Low', 'Medium', 'High'))
);

-- Alert Triggers Sensor_data (relationship)
CREATE TABLE triggers (
    alert_id INT NOT NULL,
    sensor_data_id INT NOT NULL,
    PRIMARY KEY (alert_id, sensor_data_id),
    FOREIGN KEY (alert_id) REFERENCES alert(id) ON DELETE CASCADE,
    FOREIGN KEY (sensor_data_id) REFERENCES sensor_data(id) ON DELETE CASCADE
```

```
);
```

DML:

DB seeding queries:

```
INSERT INTO users (email, password, name, role) VALUES
('admin@farm.com', 'hashed_password_1', 'Main Admin', 'admin'),
('farmer1@farm.com', 'hashed_password_2', 'Ali Farmer', 'farmer'),
('farmer2@farm.com', 'hashed_password_3', 'Ayse Farmer', 'farmer');

INSERT INTO farm (name, size, location, soil_type, planting_date, harvest_date) VALUES
('Green Valley Farm', 120.5, 'Mersin', 'Loamy', '2026-03-01', '2026-09-01'),
('Sunrise Fields', 80.0, 'Adana', 'Sandy', '2026-02-15', '2026-08-20');

INSERT INTO works_on_farm (farmer_id, farm_id) VALUES
(2, 1),
(3, 1),
(3, 2);

INSERT INTO field (farm_id, name, area, status) VALUES
(1, 'North Field', 40.0, 'Plant'),
(1, 'South Field', 60.0, 'Growth'),
(2, 'East Field', 30.0, 'Harvest');

INSERT INTO crop (name, growth_duration, local_temp, ideal_moisture) VALUES
('Wheat', 120, 22.5, 60),
('Corn', 90, 25.0, 55),
('Tomato', 75, 24.0, 70);

INSERT INTO contains (field_id, crop_id) VALUES
(1, 1),
(1, 2),
(2, 3),
(3, 2);
INSERT INTO planted_in (crop_id, field_id) VALUES
(1, 1),
(3, 2),
(2, 3);

INSERT INTO machine (name, type, status) VALUES
('Tractor A1', 'Tractor', 'working'),
```

```
('Irrigation Pump X', 'Irrigation', 'idle'),
('Harvester Z', 'Harvester', 'maintain');

INSERT INTO works_on_farm_machine (farm_id, machine_id) VALUES
(1, 1),
(1, 2),
(2, 3);

INSERT INTO works_on_field (field_id, machine_id) VALUES
(1, 1),
(2, 2),
(3, 3);

INSERT INTO iot_sensor (type, status, install_date, sensor_id) VALUES
('Temp', 'active', '2026-01-10', 'SEN-T-001'),
('Humidity', 'active', '2026-01-12', 'SEN-H-002'),
('Moist', 'inactive', '2026-01-15', 'SEN-M-003');

INSERT INTO installed_in (field_id, sensor_id) VALUES
(1, 1),
(1, 2),
(2, 3);

INSERT INTO sensor_data (value, unit, reading_time) VALUES
(23.5, 'C', '2026-05-16 10:00:00'),
(55.0, '%', '2026-05-16 10:05:00'),
(61.2, '%', '2026-05-16 10:10:00');

INSERT INTO generates (sensor_id, sensor_data_id) VALUES
(1, 1),
(2, 2),
(3, 3);

INSERT INTO alert (message, timestamp, severity) VALUES
('Temperature too high in North Field', '2026-05-16 10:15:00', 'High'),
('Humidity stable', '2026-05-16 10:20:00', 'Low');

INSERT INTO triggers (alert_id, sensor_data_id) VALUES
(1, 1),
(2, 2);
```

DML queries such as insert, update and delete are implemented in /greenav/app/api/ directories. For example, in a GET api request, we implemented the following query:

```
const query = `
      SELECT
        u.id,
        u.email,
        u.name,
        u.role,
        COUNT(wof.farm_id) as farm_count
      FROM users u
      LEFT JOIN works_on_farm wof ON u.id = wof.farmer_id
      WHERE u.role = 'farmer'
      GROUP BY u.id, u.email, u.name, u.role
      ORDER BY u.name ASC
    `;

    const result = await pool.query(query);
```

The result:

id	email	name	role	farm_count
2	farmer1@farm.com	Ali Farmer	farmer	1
3	farmer2@farm.com	Ayse Farmer	farmer	2
5	farmer@gmail.com	John Doe	farmer	1

In this section, we will include some of them with their results, but because there are many of these queries, they are not fully included in this report. You can view them all inside /greenav/app/api directory. Example of a POST API endpoint containing a query:

```
export async function POST(req: NextRequest) {
  try {
    const user = await getCurrentUser();
    if (!user || user.role !== "admin") {
      return NextResponse.json({ error: "Unauthorized - admin only" }, { status: 401 });
    }

    const { name, growth_duration, local_temp, ideal_moisture } = await req.json();

    if (!name || !growth_duration) {
```

```
      return NextResponse.json({ error: "Missing required fields" }, { status: 400 });
    }

    const result = await pool.query(
      "INSERT INTO crop (name, growth_duration, local_temp, ideal_moisture) VALUES ($1, $2, $3, $4) RETURNING *",
      [name, growth_duration, local_temp, ideal_moisture]
    );

    return NextResponse.json(result.rows[0], { status: 201 });
  } catch (err) {
    console.error(err);
    return NextResponse.json({ error: "Failed to create crop" }, { status: 500 });
  }
}
```

```
1  SELECT f.* FROM field f
2  JOIN farm fm ON f.farm_id = fm.id;
```

Results Explain Chart Export

id	farm_id	name	area	status
1	1	North Field	40.0	Plant
2	1	South Field	60.0	Growth
3	2	East Field	30.0	Harvest

```sql
SELECT  s.id,
        s.type,
        s.status,
        s.install_date,
        s.sensor_id,
        ii.field_id,
        f.name as field_name,
        f.farm_id
      FROM iot_sensor s
      LEFT JOIN installed_in ii ON s.id = ii.sensor_id
      LEFT JOIN field f ON ii.field_id = f.id
      ORDER BY s.id DESC;
```

Results Explain Chart Export Source

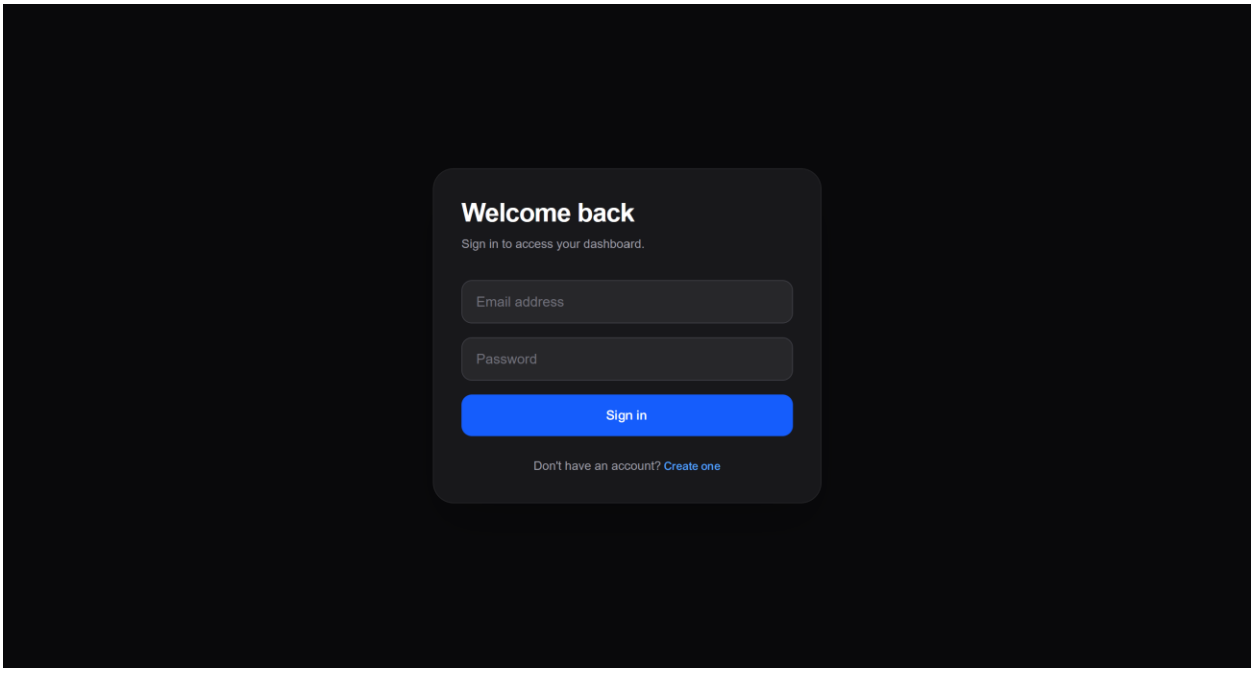
id	type	status	install_date	sensor_id	field_id	field_name	farm_id
3	Moist	inactive	2026-01-15	SEN-M-003	2	South Field	1
2	Humidity	active	2026-01-12	SEN-H-002	1	North Field	1
1	Temp	active	2026-01-10	SEN-T-001	1	North Field	1

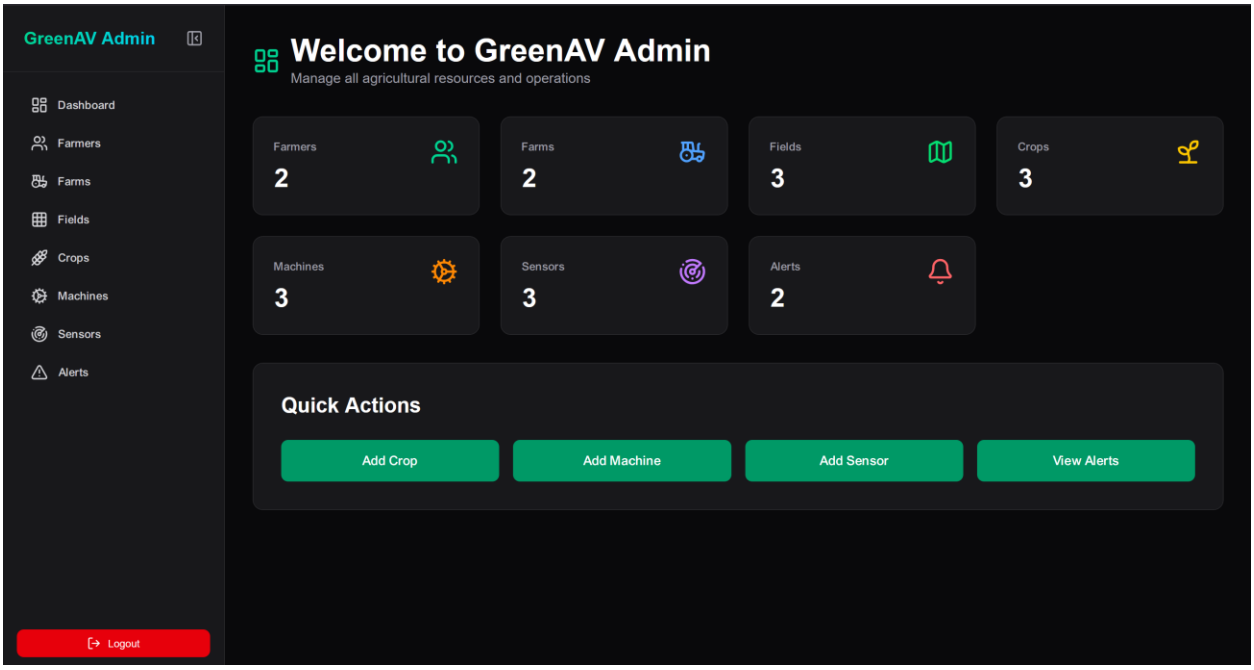
```sql
SELECT * FROM alert ORDER BY timestamp DESC;
```

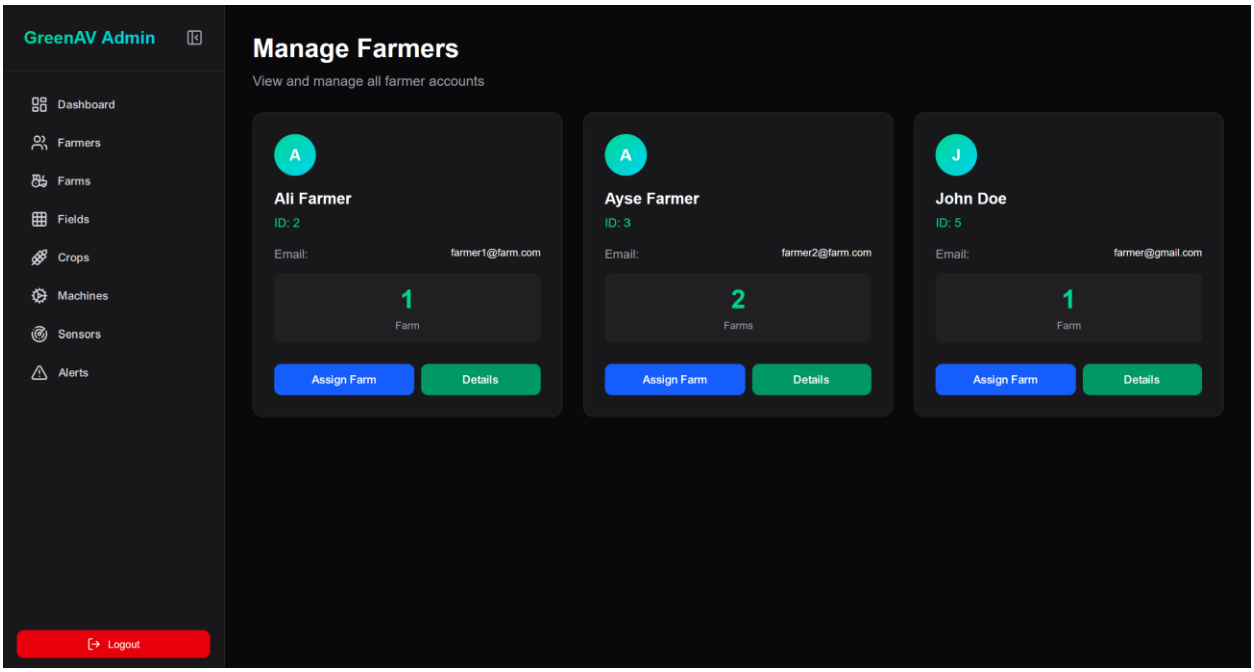
Results Explain Chart Export

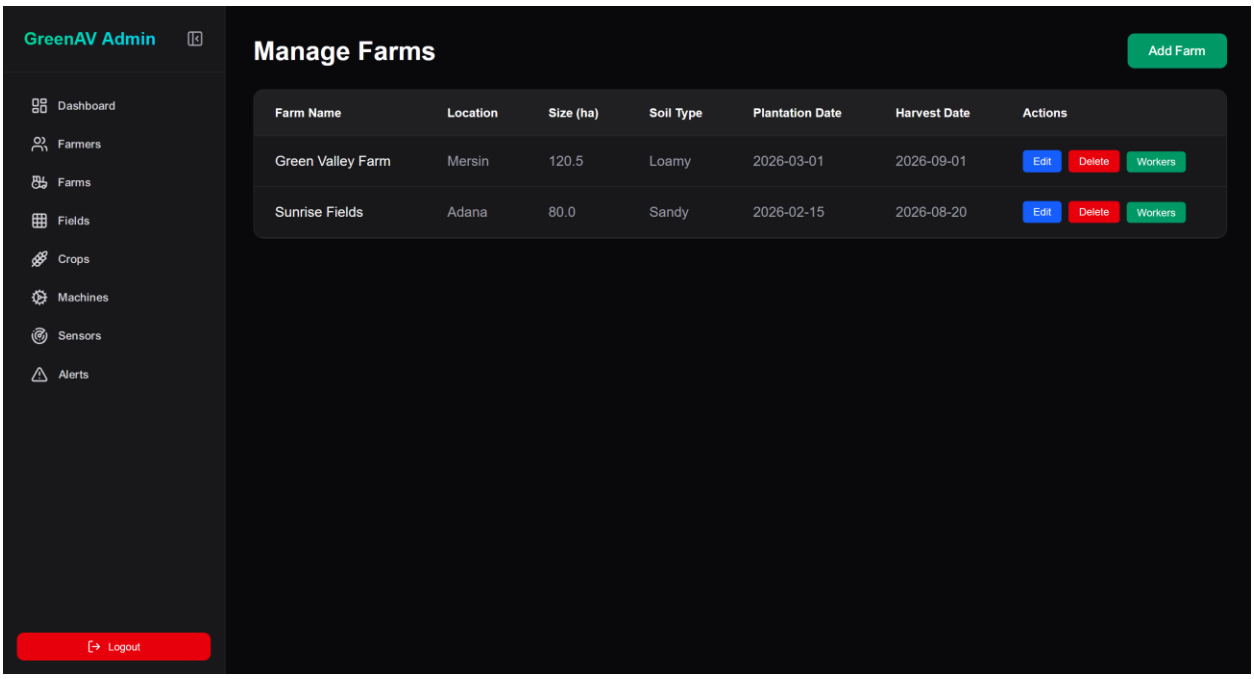
id	message	timestamp	severity
2	Humidity stable	2026-05-16 10:20:00	Low
1	Temperature too high in North Field	2026-05-16 10:15:00	High

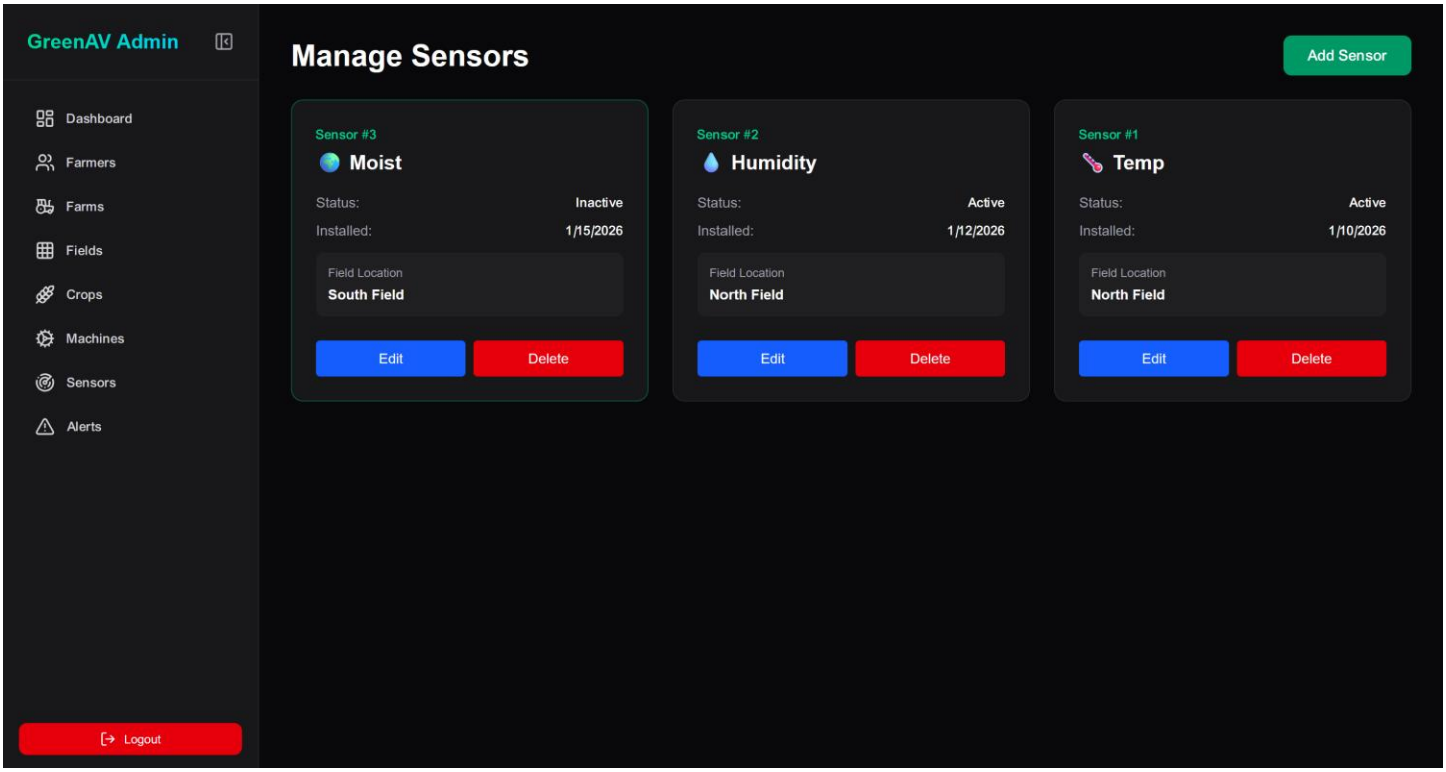
Screenshots from the UI:

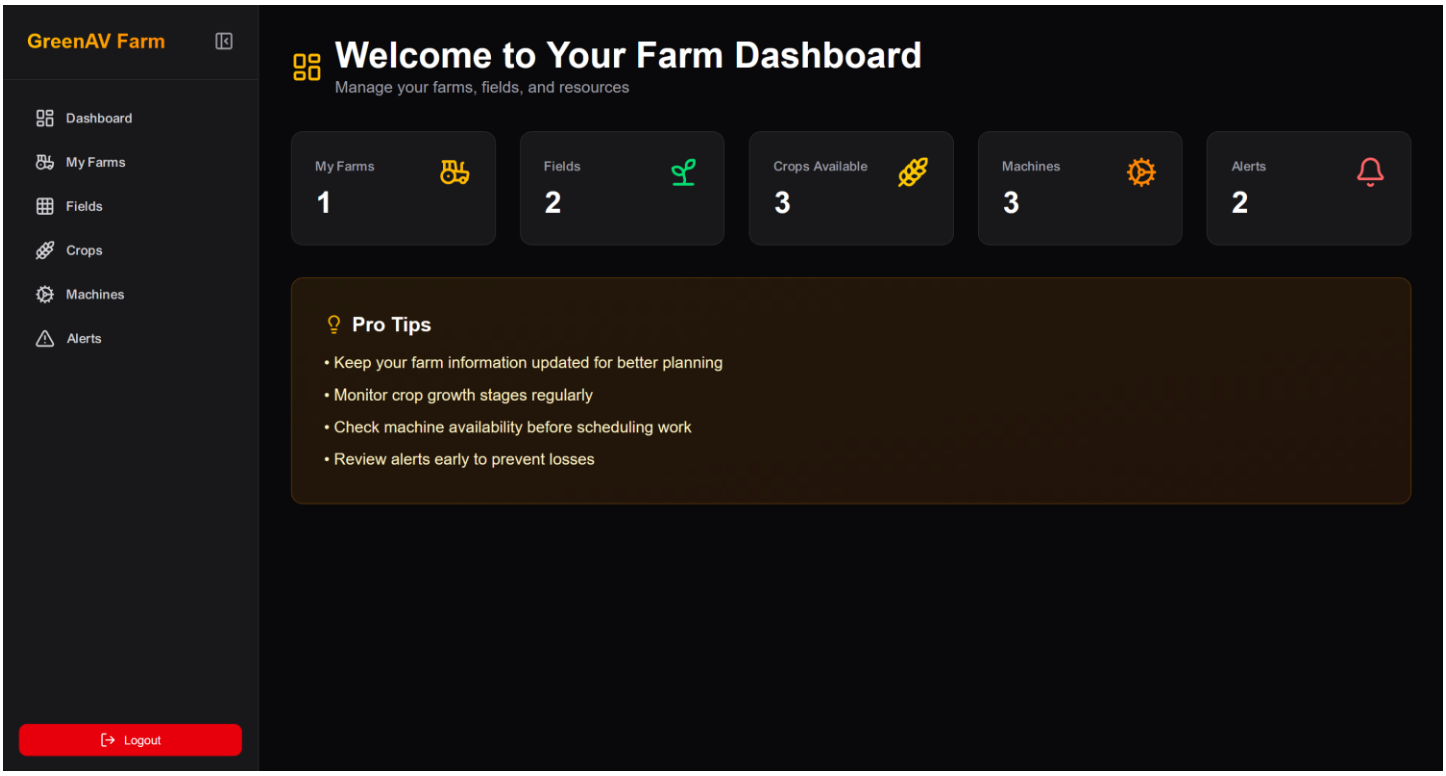

Welcome back
Sign in to access your dashboard.
Email address
Password
Sign in
Don't have an account? Create one


GreenAV Admin
Dashboard
Farmers
Farms
Fields
Crops
Machines
Sensors
Alerts
Logout
Welcome to GreenAV Admin
Manage all agricultural resources and operations
Farmers 2
Farms 2
Fields 3
Crops 3
Machines 3
Sensors 3
Alerts 2
Quick Actions
Add Crop
Add Machine
Add Sensor
View Alerts


GreenAV Admin
Dashboard
Farmers
Farms
Fields
Crops
Machines
Sensors
Alerts
Logout
Manage Farmers
View and manage all farmer accounts
A
Ali Farmer
ID: 2
Email: farmer1@farm.com
1
Farm
Assign Farm
Details
A
Ayse Farmer
ID: 3
Email: farmer2@farm.com
2
Farms
Assign Farm
Details
J
John Doe
ID: 5
Email: farmer@gmail.com
1
Farm
Assign Farm
Details


GreenAV Admin
Dashboard
Farmers
Farms
Fields
Crops
Machines
Sensors
Alerts
Logout
Manage Farms
Add Farm
Farm Name
Location
Size (ha)
Soil Type
Plantation Date
Harvest Date
Actions
Green Valley Farm
Mersin
120.5
Loamy
2026-03-01
2026-09-01
Edit
Delete
Workers
Sunrise Fields
Adana
80.0
Sandy
2026-02-15
2026-08-20
Edit
Delete
Workers


GreenAV Admin
Dashboard
Farmers
Farms
Fields
Crops
Machines
Sensors
Alerts
Logout
Manage Sensors
Add Sensor
Sensor #3
Moist
Status: Inactive
Installed: 1/15/2026
Field Location
South Field
Edit
Delete
Sensor #2
Humidity
Status: Active
Installed: 1/12/2026
Field Location
North Field
Edit
Delete
Sensor #1
Temp
Status: Active
Installed: 1/10/2026
Field Location
North Field
Edit
Delete


GreenAV Farm
Dashboard
My Farms
Fields
Crops
Machines
Alerts
Logout
Welcome to Your Farm Dashboard
Manage your farms, fields, and resources
My Farms
1
Fields
2
Crops Available
3
Machines
3
Alerts
2
Pro Tips
• Keep your farm information updated for better planning
• Monitor crop growth stages regularly
• Check machine availability before scheduling work
• Review alerts early to prevent losses

GreenAV Farm

- Dashboard
- My Farms
- Fields
- Crops
- Machines
- Alerts

Logout

Farms I Work On

View all farms assigned to you

Green Valley Farm

Mersin

Size:	120.5 ha
Soil Type:	Loamy
Planting Date:	3/1/2026
Harvest Date:	9/1/2026

View Details

GreenAV Farm

- Dashboard
- My Farms
- Fields
- Crops
- Machines
- Alerts

Logout

My Fields

View all fields in your assigned farms

North Field

Green Valley Farm

Area:	40.0 ha

STATUS: PLANT

View Details

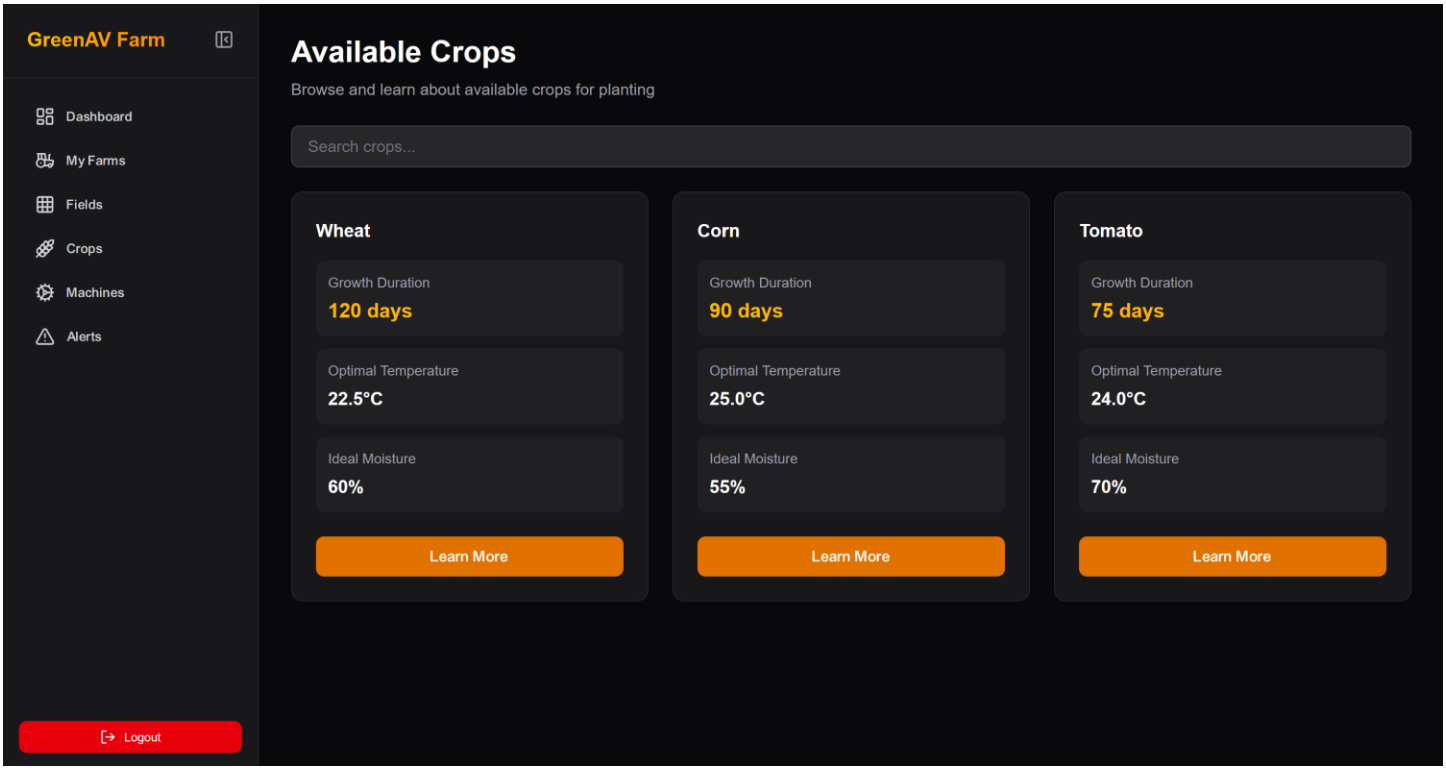
South Field

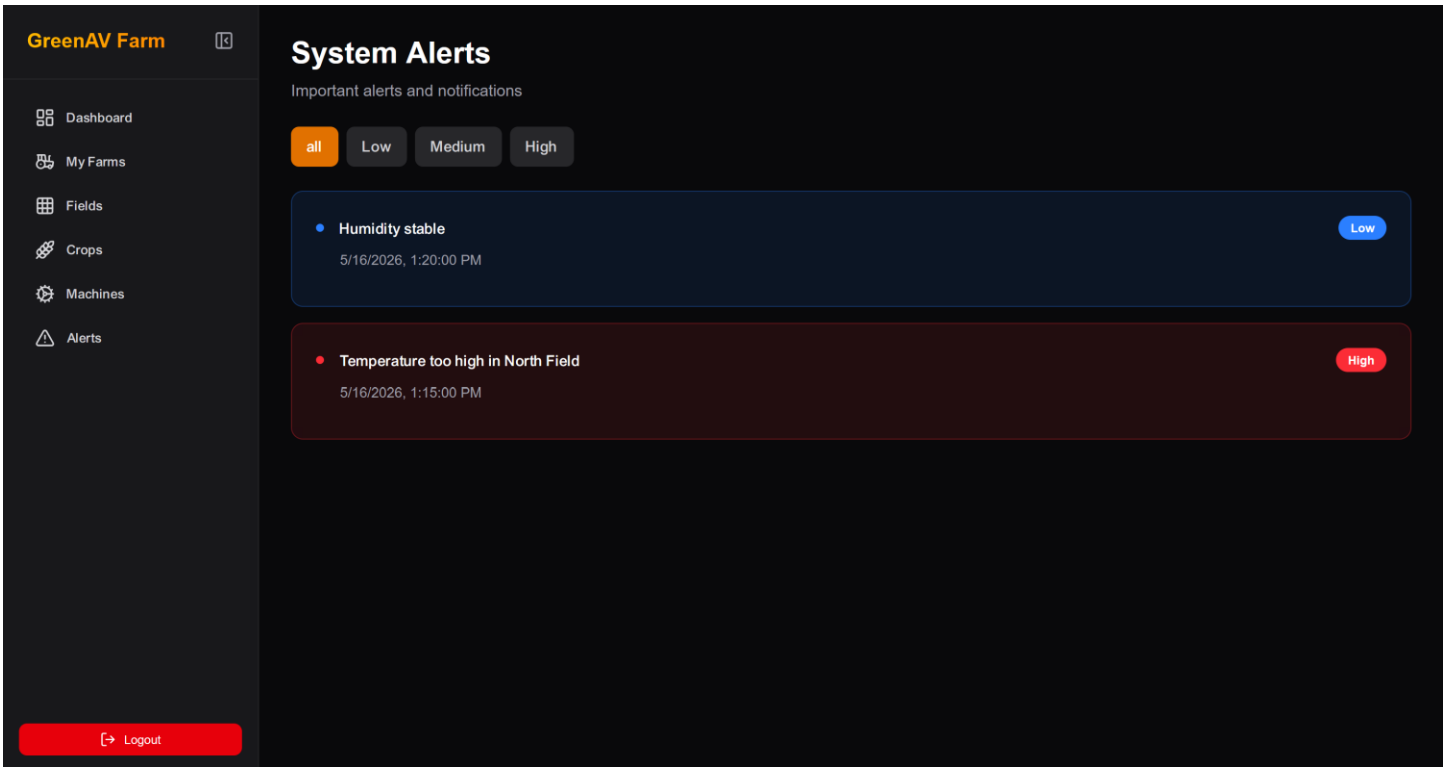
Green Valley Farm

Area:	60.0 ha

STATUS: GROWTH

View Details





Functions, triggers and procedures:

```
-- sensor alert trigger
-- Function: Create alerts automatically when sensor data is inserted
CREATE OR REPLACE FUNCTION check_sensor_alert()
RETURNS TRIGGER AS $$
```

```
DECLARE
    sensor_type TEXT;
    alert_message TEXT;
    alert_severity TEXT;
    new_alert_id INT;
BEGIN
    -- Find the sensor type that generated this sensor_data row
    SELECT s.type
    INTO sensor_type
    FROM generates g
    JOIN iot_sensor s ON s.id = g.sensor_id
    WHERE g.sensor_data_id = NEW.id
    LIMIT 1;

    -- If no sensor is linked yet, do nothing
    IF sensor_type IS NULL THEN
        RETURN NEW;
    END IF;

    -- Temperature sensor rules
    IF sensor_type = 'Temp' THEN
        IF NEW.value > 35 THEN
            alert_message := 'High temperature detected: ' || NEW.value || ' °C';
            alert_severity := 'High';
        ELSIF NEW.value < 5 THEN
            alert_message := 'Low temperature detected: ' || NEW.value || ' °C';
            alert_severity := 'Medium';
        END IF;
    END IF;

    -- Humidity sensor rules
    IF sensor_type = 'Humidity' THEN
        IF NEW.value < 30 THEN
            alert_message := 'Low humidity detected: ' || NEW.value || ' %';
            alert_severity := 'Medium';
        ELSIF NEW.value > 90 THEN
            alert_message := 'High humidity detected: ' || NEW.value || ' %';
            alert_severity := 'Low';
        END IF;
    END IF;

    -- Soil moisture sensor rules
```

```
    IF sensor_type = 'Moist' THEN
        IF NEW.value < 25 THEN
            alert_message := 'Critical low soil moisture: ' || NEW.value || ' %';
            alert_severity := 'High';
        ELSIF NEW.value < 40 THEN
            alert_message := 'Low soil moisture detected: ' || NEW.value || ' %';
            alert_severity := 'Medium';
        ELSIF NEW.value > 85 THEN
            alert_message := 'Excessive soil moisture detected: ' || NEW.value || ' %';
            alert_severity := 'Low';
        END IF;
    END IF;

    -- If an alert condition was met, create the alert and relationships
    IF alert_message IS NOT NULL THEN
        -- Insert into alert table
        INSERT INTO alert (message, timestamp, severity)
        VALUES (alert_message, NEW.reading_time, alert_severity)
        RETURNING id INTO new_alert_id;

        -- Insert into alert_trigger table
        INSERT INTO alert_trigger (alert_id)
        VALUES (new_alert_id);

        -- Link trigger to the sensor data that caused it
        INSERT INTO trigger_sensor (trigger_id, sensor_data_id)
        VALUES (new_alert_id, NEW.id);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger: Runs after a new relationship is created between sensor and sensor_data
CREATE OR REPLACE TRIGGER trg_check_sensor_alert
AFTER INSERT ON generates
FOR EACH ROW
EXECUTE FUNCTION check_sensor_alert();
```

Results Explain Chart Export

Success. No rows returned

```
-- Automatically update machine status to 'working'
--    when a machine is assigned to a field
CREATE OR REPLACE FUNCTION set_machine_working()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE machine
    SET status = 'working'
    WHERE id = NEW.machine_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_machine_working
AFTER INSERT ON works_on_field
FOR EACH ROW
EXECUTE FUNCTION set_machine_working();
```

Results Explain Chart Export

Success. No rows returned

```
-- Automatically update field status to 'Growth'
--    when a crop is planted in the field
CREATE OR REPLACE FUNCTION set_field_growth()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE field
    SET status = 'Growth'
    WHERE id = NEW.field_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_set_field_growth
AFTER INSERT ON planted_in
FOR EACH ROW
EXECUTE FUNCTION set_field_growth();
```

Results Explain Chart Export

Success. No rows returned

```
-- Prevent duplicate user emails (custom error message)
CREATE OR REPLACE FUNCTION check_duplicate_email()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM users
        WHERE email = NEW.email
          AND id <> COALESCE(NEW.id, 0)
    ) THEN
        RAISE EXCEPTION 'Email % is already registered.', NEW.email;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_duplicate_email
BEFORE INSERT OR UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION check_duplicate_email();
```

Results Explain Chart Export

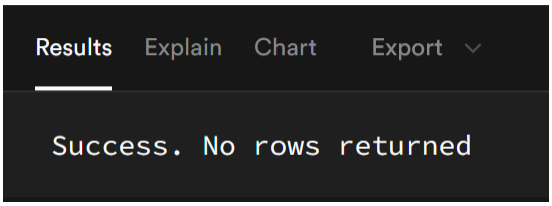
Success. No rows returned

```
-- Automatically set field status to 'Harvest'
```

```
--    when the farm's harvest date is reached
CREATE OR REPLACE FUNCTION update_field_harvest_status()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.harvest_date IS NOT NULL
       AND NEW.harvest_date <= CURRENT_DATE THEN
        UPDATE field
        SET status = 'Harvest'
        WHERE farm_id = NEW.id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_field_harvest_status
AFTER INSERT OR UPDATE ON farm
FOR EACH ROW
EXECUTE FUNCTION update_field_harvest_status();
```



DB connection:

.env:

DATABASE_URL=postgresql://postgres.pwodfbjkngieomarodaa:yourtherichboy1@aws-1-ap-northeast-1.pooler.supabase.com:5432/postgres

/lib/db.ts:

```
import { Pool } from "pg";
const connectionString = process.env.DATABASE_URL;
if (!connectionString) {
  throw new Error("DATABASE_URL is not defined in environment variables");
}
```

```
export const pool = new Pool({
  connectionString,
});
```